The Design and Formal Analysis of the Merkle-Chained Evidence Ledger

John G. Underhill
Quantum Resistant Cryptographic Solutions Corporation

Abstract

Merkle-Chained Evidence Ledger (MCEL) is an append-only cryptographic ledger construction designed to provide integrity, ordering, and auditability guarantees for event-based systems without requiring consensus, proof-of-work, or replicated global state. MCEL represents records as typed, canonically encoded objects, derives payload and record commitments using domain-separated cSHAKE-256, aggregates ordered record commitments into block Merkle roots, and links sealed checkpoints through a previous-checkpoint commitment. The resulting construction provides deterministic verification of record integrity, block membership, checkpoint continuity, and externally retained anchor commitments.

This paper gives a formal definition of MCEL and analyzes its security under standard collision-resistance, second-preimage-resistance, and signature-unforgeability assumptions. The analysis models the reference block-and-checkpoint commitment architecture, the two-level payload and record commitment structure, cSHAKE-based domain separation, unsigned baseline anchor commitments, an always-signed reference checkpoint bundle, optional signed-anchor profiles, inclusion proofs, consistency proofs, key-rotation records, and policy enforcement as an operational layer. The main results establish record binding, Merkle-root binding, checkpoint-chain prefix uniqueness, tamper evidence, truncation detectability relative to known checkpoints, and conditional authenticity for signed attestations.

Contents

1 Introduction

Audit trails and event logs are central to security-critical systems, including compliance infrastructures, evidence retention systems, software supply-chain provenance systems, regulated institutional records, and forensic chain-of-custody systems. In these environments, it is often necessary to demonstrate that recorded events have not been altered, deleted, or reordered after being accepted into an operational record. Conventional database logs provide useful operational history, but they generally do not provide portable cryptographic evidence that a third party can validate independently of the database operator. Public blockchain systems provide stronger public ordering properties, but they also introduce consensus assumptions, global replication requirements, economic mechanisms, and operational complexity that are unnecessary for many controlled-writer audit settings.

MCEL is designed for a narrower and more analyzable problem: the construction of an append-only, cryptographically verifiable evidence ledger under a single-writer or controlled-writer model. The writer is responsible for generating records and sealing checkpoints. Verifiers may be external parties that do not trust the storage medium, the transport channel, or the operator's presentation of historical data. The construction therefore emphasizes deterministic verification, canonical serialization, and explicit commitment structure rather than distributed agreement.

The ledger is organized in layers. First, each record payload is committed under a payload domain, distinguishing plaintext payload commitments from ciphertext payload commitments. Second, a record commitment binds the canonical record header to the payload commitment. Third, a deterministic Merkle tree aggregates the ordered record commitments of a block into a block Merkle root. Fourth, a checkpoint commitment binds the checkpoint header, the block Merkle root, and the previous checkpoint commitment. The checkpoint commitment chain provides the cumulative ledger state. Anchors bind checkpoint commitments to external references. In the baseline construction, anchors are hash commitments and provide integrity but not authenticity; authenticity is obtained only when an external or profile-defined signature is applied.

The principal contribution of this paper is a formal model of the MCEL construction and a proof-oriented security analysis. The analysis is intentionally scoped. It establishes tamper evidence and prefix uniqueness relative to known checkpoint commitments; it does not claim consensus, liveness, truthfulness of record contents, or prevention of equivocation by a malicious authority. MCEL makes equivocation detectable when verifiers compare checkpoints, signed attestations, or externally retained anchors. It does not prevent an authorized writer from producing false records at the time of append, and it does not define the surrounding governance rules that determine who is allowed to append records.

The paper is organized as follows. Section 2 introduces notation and cryptographic primitives. Section 3 defines the system and adversary model. Section 4 describes the MCEL construction. Section 5 gives the formal syntax and algorithms. Section 6 proves the main security properties. Sections 7 and 8 state operational invariants and correctness properties. Sections 9, 10, and 11 discuss non-goals, implementation guidance, and related work.

2 Preliminaries and Notation

2.1 Byte Strings and Encodings

All objects in MCEL are ultimately represented as byte strings. Let $\{0,1\}^*$ denote the set of finite byte strings, and let $|X|$ denote the length of a byte string X in bytes unless otherwise stated. Concatenation is written as $X \| Y$. The function $\mathsf{Enc}(\cdot)$ denotes canonical serialization of a structured

object to bytes.

Canonical serialization is injective over the object class being encoded. If A and B are distinct structured objects of the same type, then $\mathsf{Enc}(A) \neq \mathsf{Enc}(B)$. All fixed-width integer fields in MCEL encodings are interpreted as big-endian byte strings. All hash outputs considered in this paper have length $\lambda = 256$ bits, or 32 bytes.

2.2 cSHAKE-256 and Domain Separation

MCEL uses cSHAKE-256 as defined in NIST SP 800-185, derived from the SHA-3 family specified in FIPS 202. The function-name string is fixed as

$$N = \texttt{MCEL-LEDGER}.$$

Domain separation is provided by the cSHAKE customization string S. For readability, we write

$$\mathsf{cSHAKE256}_S(X) := \mathsf{cSHAKE256}(X, 256, N, S),$$

where the output length is 256 bits. Distinct customization strings define distinct MCEL hash domains. The relevant domains are listed in Table 1.

Symbol	`Customization string`	Role	Input class
BLCK	`MCEL-DOMAIN-BLCK`	Block commitment	Encoded block header and block Merkle root
CHCK	`MCEL-DOMAIN-CHCK`	Checkpoint commitment	Encoded checkpoint header, block root, previous checkpoint commitment
CTXT	`MCEL-DOMAIN-CTXT`	Ciphertext payload commitment	Encrypted payload bytes
NODE	`MCEL-DOMAIN-NODE`	Merkle internal node and empty-root constant domain	Pair of child hashes, or the implementation-defined empty-root input
PTXT	`MCEL-DOMAIN-PTXT`	Plaintext payload commitment	Plaintext payload bytes
RCRD	`MCEL-DOMAIN-RCRD`	Record commitment	Encoded record header and payload commitment
ANCR	`MCEL-DOMAIN-ANCR`	Anchor commitment	Checkpoint commitment and encoded anchor reference

Table 1: MCEL cSHAKE-256 domain labels.

The security analysis treats the domain-separated cSHAKE calls as independent fixed-output hash functions for their respective structured inputs. The proofs rely on collision resistance and second-preimage resistance in these domains, and on the absence of ambiguity in canonical encodings.

2.3 Signatures

MCEL checkpoint bundles use a digital signature over the checkpoint commitment in the reference profile. The default reference signature primitive is ML-DSA, standardized in FIPS 204; the implementation also provides an algorithm-agility path for an alternate hash-based signature profile. In this paper, a signature scheme is denoted by $(\mathsf{KeyGen}, \mathsf{Sign}, \mathsf{Verify})$. The security assumption is existential unforgeability under chosen-message attack (EUF-CMA) for the selected signature

scheme and parameter set. Signature use is orthogonal to the core hash-binding construction: unsigned commitments provide integrity binding, while signatures provide authenticity relative to the verification key.

2.4 Quantum Model

MCEL is analyzed primarily in the standard post-quantum, classical-oracle model, denoted here as the Q1 model. In this model the adversary may run a quantum computation against public parameters and public artifacts, but it interacts with protocol interfaces, signing oracles, storage interfaces, and verification algorithms through classical inputs and outputs. The assumed signature scheme is required to remain EUF-CMA secure against such quantum polynomial-time adversaries, and the domain-separated cSHAKE-256 calls are assumed to provide the stated collision-resistance and second-preimage-resistance properties against quantum adversaries at the selected output length.

The stronger Q2 model, in which an adversary may make quantum superposition queries to hash, signing, or verification oracles, is not claimed by this paper. A Q2 treatment would require a dedicated quantum-random-oracle or quantum-access model for the cSHAKE domains and for any signature oracle used by a deployment profile. Thus, unless a theorem explicitly states otherwise, the security statements are Q1 statements with classical protocol access and quantum computation.

2.5 Notation

The following notation is used throughout the paper.

Notation	Meaning
P_i	Payload bytes of record i
Hdr_i	Record header of record i
R_i	Record i, consisting of (Hdr_i, P_i)
PCom_i	Payload commitment of record i
RCom_i	Record commitment of record i
B_j	Block j
BHdr_j	Header of block j
BRoot_j	Merkle root over record commitments in block j
BCom_j	Block commitment of block j
Q_j	Checkpoint j
QHdr_j	Checkpoint header of checkpoint j
QCom_j	Checkpoint commitment of checkpoint j
ARef	Encoded anchor reference
ACom	Anchor commitment

A negligible function in the security parameter is written $\mathrm{negl}(\lambda)$.

3 System and Threat Model

3.1 System Participants

The MCEL model contains three principal roles.

The *ledger authority* constructs records, appends them to storage, seals blocks, and produces checkpoint bundles. It controls any signing keys used for checkpoint signatures or optional signed anchor profiles.

A *storage provider* retains serialized records, sealed blocks, checkpoint bundles, head checkpoint objects, and optional auxiliary artifacts. The storage provider is not a trust anchor. MCEL treats stored bytes as adversarial until verified.

A *verifier* receives records, blocks, checkpoints, proofs, or anchors and checks them cryptographically. A verifier may be an auditor, regulator, relying application, or automated validation process.

3.2 Trust Assumptions

MCEL assumes that cryptographic primitives satisfy their stated security properties. Public verification keys are assumed to be authentic for the ledger authority when signatures are used. The storage layer may be malicious, unreliable, or stale. Network delivery may be delayed, reordered, or suppressed.

The ledger authority may be honest at record-generation time, malicious from the beginning, or compromised after some time. MCEL does not establish the truthfulness of records at the moment they are generated. It establishes that accepted records, once cryptographically committed into sealed blocks and checkpoint chains, cannot be modified, deleted, or reordered without detection relative to a known checkpoint or anchor.

3.3 Adversary Capabilities

The adversary is modeled as a probabilistic polynomial-time algorithm with the following capabilities.

- Read, copy, modify, delete, reorder, or substitute stored records, sealed blocks, checkpoint bundles, and anchor references.
- Present different ledger histories to different verifiers.
- Replay older valid checkpoints or anchors.
- Delay or suppress delivery of records, checkpoints, and anchors.
- Compromise the ledger authority after some point in time, including access to signing keys used after compromise.

The adversary is not assumed to break cSHAKE-256, find collisions in the relevant domain-separated functions, or forge signatures under uncompromised signing keys.

3.4 Out-of-Scope Threats

The following are outside the cryptographic scope of this analysis.

- Denial-of-service attacks against the ledger authority, storage provider, or verifier.
- Side-channel attacks against implementations of cSHAKE-256, ML-DSA, or auxiliary primitives.
- Operational failure to protect signing keys.
- False or misleading records intentionally created by an authorized writer.
- Consensus failure, since MCEL is not a consensus protocol.

These exclusions do not make the threats unimportant. They define the boundary of the formal claims.

4 MCEL Construction

4.1 Layered Structure

MCEL is a layered commitment system. The layers are:

1. payload commitment;
2. record commitment;
3. block Merkle aggregation;
4. block commitment;
5. checkpoint commitment chain;
6. optional signature and anchor layers.

The key design point is that the cumulative ledger state is not a per-record hash chain. Individual record commitments do not include the previous ledger commitment. Cumulative state is provided at checkpoint granularity: each checkpoint commitment includes the previous checkpoint commitment. Records are bound to one another inside a block by the ordered Merkle root, and blocks are bound over time by the checkpoint chain.

4.2 Record Types

MCEL records are typed. The baseline type set contains checkpoint-reference records, event records, key-rotation records, and policy or configuration records. Event records carry application data. Key-rotation records commit a new verification key or key identifier transition into the ledger. Policy records may record configuration state. Anchor data is not required to be a ledger record; the baseline anchor mechanism is an external commitment binding a checkpoint commitment to an encoded external reference.

4.3 Payload and Record Commitments

A payload commitment is computed over the record payload bytes. The payload domain depends on whether the payload is committed as plaintext or ciphertext. This distinction prevents accidental substitution between plaintext and ciphertext payload interpretations.

A record commitment binds the canonical record header to the payload commitment. The record header contains sequence, timestamp, payload length, record type, flags, version, and a 32-byte key identifier. The canonical encoded record header length is 58 bytes. The namespace identifier used by a ledger instance is operational context; the cryptographic domain separation in the commitment functions is provided by the cSHAKE customization strings.

4.4 Block Merkle Root and Block Commitment

A block contains an ordered list of record commitments. The block Merkle root is computed deterministically using the node domain. If a level of the tree has an odd number of nodes, the last node is hashed with itself. This left-duplicate rule is normative for MCEL tree construction.

The block commitment binds the encoded block header and the block Merkle root. The encoded block header length is 62 bytes. The block commitment is a compact identifier for the sealed block and can be stored with the block artifact.

4.5 Checkpoint Commitment Chain

A checkpoint binds an encoded checkpoint header, the sealed block Merkle root, and the previous checkpoint commitment. The encoded checkpoint header length is 62 bytes. The first checkpoint uses a distinguished previous commitment value 0^{256}. Subsequent checkpoints include the previous checkpoint commitment, creating a cumulative chain.

The checkpoint commitment is the principal ledger-state identifier. The pair consisting of the checkpoint commitment and the checkpoint header metadata identifies the sealed state of the ledger at the checkpoint boundary. The reference construction signs every checkpoint bundle: checkpoint sealing produces a signature over the checkpoint commitment, and bundle, head, and audit-path verification require a valid signature under the selected verification key. In the default profile this signature is ML-DSA; alternate signature profiles are analyzed by substituting their EUF-CMA advantage. The hash-binding of the checkpoint chain is nonetheless independent of the signature; the unsigned commitment alone provides integrity binding, while the mandatory bundle signature provides authenticity.

4.6 Anchors

An anchor commitment binds a checkpoint commitment to an encoded anchor reference. The anchor reference contains a version, flags, type field, reserved byte, chain identifier length, reference length, chain identifier, and reference bytes. The baseline anchor commitment is an unsigned hash commitment. It provides binding integrity: a verifier holding a specific anchor commitment cannot be given a different checkpoint-reference pair without a collision. It does not authenticate the ledger authority. Authenticity requires an optional signed-anchor profile or publication in an authenticated external system.

4.7 Policy Layer

MCEL may include an operational policy layer. Such a layer can enforce payload-size bounds, allowed record-type masks, mandatory encryption, monotonic record sequence numbers, monotonic timestamps, and key-identifier linkage across checkpoints. These checks support deployment correctness but are not the basis of the core cryptographic binding proofs. If a policy layer is absent or misconfigured, the resulting ledger may violate deployment policy, but the cryptographic commitments still bind the records and checkpoints that were actually produced.

5 Formal Definition of MCEL

5.1 Canonical Headers

Let Hdr_i be the record header of record R_i. Its canonical encoding is

$$\mathsf{Enc}(\mathsf{Hdr}_i) = \mathsf{seq}_i \parallel \mathsf{time}_i \parallel \mathsf{plen}_i \parallel \mathsf{type}_i \parallel \mathsf{flags}_i \parallel \mathsf{ver}_i \parallel \mathsf{keyid}_i,$$

where the field lengths are 8, 8, 4, 4, 1, 1, and 32 bytes respectively, and integer fields are big-endian. Thus $|\mathsf{Enc}(\mathsf{Hdr}_i)| = 58$ bytes.

Let BHdr_j be a block header. Its encoded form is

$$\mathsf{Enc}(\mathsf{BHdr}_j) = \mathsf{bseq}_j \parallel \mathsf{first}_j \parallel \mathsf{time}_j \parallel \mathsf{rcount}_j \parallel \mathsf{flags}_j \parallel \mathsf{ver}_j \parallel \mathsf{keyid}_j,$$

with field lengths 8, 8, 8, 4, 1, 1, and 32 bytes. Thus $|\mathsf{Enc}(\mathsf{BHdr}_j)| = 62$ bytes.

Let QHdr_j be a checkpoint header. Its encoded form is

$$\mathsf{Enc}(\mathsf{QHdr}_j) = \mathsf{qseq}_j \parallel \mathsf{first}_j \parallel \mathsf{time}_j \parallel \mathsf{rcount}_j \parallel \mathsf{flags}_j \parallel \mathsf{ver}_j \parallel \mathsf{keyid}_j,$$

with the same 62-byte field layout as the block header, but with checkpoint semantics.

5.2 Payload Commitment

Let $P_i \in \{0,1\}^*$ be the payload of record R_i. Let $e_i \in \{0,1\}$ indicate whether the payload is committed as plaintext or ciphertext. Define

$$\mathsf{PCom}_i = \begin{cases} \mathsf{cSHAKE256}_{\mathsf{PTXT}}(P_i)\,, & e_i = 0, \\ \mathsf{cSHAKE256}_{\mathsf{CTXT}}(P_i)\,, & e_i = 1. \end{cases}$$

The payload commitment has length 32 bytes. The distinction between PTXT and CTXT is part of the commitment domain and prevents cross-interpretation of payload state. In the reference implementation the payload is non-empty ($P_i \neq \epsilon$); zero-length payloads are rejected at commitment time.

5.3 Record Commitment

The record commitment of $R_i = (\mathsf{Hdr}_i, P_i)$ is

$$\mathsf{RCom}_i = \mathsf{cSHAKE256}_{\mathsf{RCRD}}(\mathsf{Enc}(\mathsf{Hdr}_i) \parallel \mathsf{PCom}_i)\,.$$

No previous ledger commitment is an input to RCom_i. The record commitment is a stateless binding of the record header and the payload commitment.

5.4 Merkle Node and Empty Root

For two 32-byte node values x and y, define

$$\mathsf{Node}(x, y) = \mathsf{cSHAKE256}_{\mathsf{NODE}}(x \parallel y)\,.$$

For the zero-leaf case, MCEL defines an implementation constant

$$\mathsf{EmptyRoot} = \mathsf{EmptyRoot}_{\mathsf{NODE}},$$

where $\mathsf{EmptyRoot}_{\mathsf{NODE}}$ is the 32-byte value returned by the reference empty-root routine for the NODE domain. This value is deterministic and domain-separated, but it is not used on the ordinary sealing path because sealed blocks contain at least one record. Non-empty Merkle trees use only the node-compression rule below.

For a non-empty sequence of leaves $(x_0, \ldots, x_{n-1})$, $\mathsf{MerkleRoot}(x_0, \ldots, x_{n-1})$ is computed level by level. Adjacent pairs are compressed as $\mathsf{Node}(x_{2t}, x_{2t+1})$. If the number of nodes at a level is odd, the last node z is compressed as $\mathsf{Node}(z, z)$. The final node is the root.

5.5 Block Root and Block Commitment

Let block B_j contain the ordered record commitment sequence

$$\mathcal{R}_j = (\mathsf{RCom}_{a_j}, \ldots, \mathsf{RCom}_{b_j}).$$

The block Merkle root is

$$\mathsf{BRoot}_j = \mathsf{MerkleRoot}(\mathcal{R}_j).$$

The block commitment is

$$\mathsf{BCom}_j = \mathsf{cSHAKE256}_{\mathsf{BLCK}}(\mathsf{Enc}(\mathsf{BHdr}_j) \parallel \mathsf{BRoot}_j)\,.$$

5.6 Checkpoint Commitment

Let $\mathsf{QCom}_0 = 0^{256}$ be the genesis previous-checkpoint value. For checkpoint Q_j with header QHdr_j and associated block root BRoot_j, define

$$\mathsf{QCom}_j = \mathsf{cSHAKE256}_{\mathsf{CHCK}}\big(\mathsf{Enc}(\mathsf{QHdr}_j) \parallel \mathsf{BRoot}_j \parallel \mathsf{QCom}_{j-1}\big)\,.$$

If signatures are enabled, the signed message is QCom_j. The signature does not change QCom_j; it authenticates it.

5.7 Anchor Commitment

An anchor reference is encoded as

$$\mathsf{Enc}(\mathsf{ARef}) = \mathsf{ver} \parallel \mathsf{flags} \parallel \mathsf{type} \parallel \mathsf{reserved} \parallel \mathsf{chainlen} \parallel \mathsf{reflen} \parallel \mathsf{chainid} \parallel \mathsf{ref},$$

where the first four fields are one byte each, the length fields are two-byte big-endian unsigned integers, and the variable fields have the declared lengths.

The anchor commitment binding checkpoint commitment QCom_j to the encoded reference is

$$\mathsf{ACom}_j = \mathsf{cSHAKE256}_{\mathsf{ANCR}}\big(\mathsf{QCom}_j \parallel \mathsf{Enc}(\mathsf{ARef})\big)\,.$$

This is a hash commitment, not a signature.

5.8 Inclusion Proofs

An inclusion proof for leaf x_i in a block with root M and leaf count n consists of the target leaf x_i, the target index i, the leaf count n, a sequence of sibling hashes $(s_0, \ldots, s_{d-1})$, and a sequence of direction bits $(b_0, \ldots, b_{d-1})$. The accumulator is initialized as $a_0 = x_i$. For level t, define

$$a_{t+1} = \begin{cases} \mathsf{Node}(a_t, s_t), & b_t = 0, \\ \mathsf{Node}(s_t, a_t), & b_t = 1. \end{cases}$$

The verifier derives each direction bit b_t from the binary expansion of the index i, namely $b_t = \lfloor i/2^t \rfloor \bmod 2$, rather than trusting a value supplied in the proof, and recomputes the path length d from the leaf count n. The proof verifies if $a_d = M$, the count n is consistent with the recomputed path length, and the supplied directions, if present, agree with those determined by i. Because the directions are fixed by i, the accumulator path binds the leaf to position i.

The reference structured proof object additionally carries a format version, a generation timestamp, the asserted record position, the ledger record count, and the committed root. The version and timestamp fields are serialized proof metadata; they are checked for format validity where applicable but are not part of the Merkle root M. A conforming verifier MUST derive the path directions from the asserted position, or equivalently reject any proof whose supplied directions are inconsistent with it, so that the asserted position is cryptographically bound rather than merely advisory.

5.9 Consistency Proofs

MCEL may provide a deterministic checkpoint consistency proof between an older block root and a newer block root by supplying the complete ordered leaf commitment sequence of the newer tree. For counts $m \le n$, verification recomputes

$$M_m = \mathsf{MerkleRoot}(x_0, \ldots, x_{m-1})$$

and

$$M_n = \mathsf{MerkleRoot}(x_0, \ldots, x_{n-1}).$$

The proof is accepted if M_m equals the asserted older root and M_n equals the asserted newer root. This proof format is not compact, but it is deterministic and exactly matches the duplicate-last Merkle construction.

This consistency proof applies to a *cumulative* leaf sequence: the newer leaf set $x_0, \ldots, x_{n-1}$ must contain the older set $x_0, \ldots, x_{m-1}$ as a prefix. In the block-and-checkpoint construction each block root commits an independent ordered segment, so two checkpoints stand in a consistency relationship only when the deployment seals cumulative block trees, or when consistency is evaluated over the underlying record-commitment sequence rather than across disjoint block roots. The verification procedure establishes the prefix relationship for any such nested pair; it does not by itself establish nesting between arbitrary block roots.

6 Security Analysis

6.1 Assumptions

Collision resistance. For each MCEL domain S, the function $X \mapsto \mathsf{cSHAKE256}_S(X)$ is collision resistant. For every PPT adversary $\mathcal{A}$,

$$\Pr[(X, X') \leftarrow \mathcal{A}(1^\lambda) : X \neq X' \wedge \mathsf{cSHAKE256}_S(X) = \mathsf{cSHAKE256}_S\big(X'\big)] \leq \mathrm{negl}(\lambda)\,.$$

Second-preimage resistance. For each MCEL domain S, given X, it is infeasible to find $X' \neq X$ such that $\mathsf{cSHAKE256}_S(X) = \mathsf{cSHAKE256}_S(X')$.

Domain separation. Distinct customization strings define independent MCEL hash domains. A successful substitution between two different object classes that preserves the same commitment value implies either a cross-domain collision or a violation of the cSHAKE domain-separation assumption.

Signature unforgeability. When checkpoint or anchor signatures are used, the signature scheme is EUF-CMA secure.

6.2 Canonical Encoding

Definition 1 (Canonical encoding)**.** *For an object class $\mathcal{C}$, an encoding function $\mathsf{Enc} : \mathcal{C} \to \{0,1\}^*$ is canonical if for all $A, B \in \mathcal{C}$, $A \neq B$ implies $\mathsf{Enc}(A) \neq \mathsf{Enc}(B)$.*

MCEL relies on fixed-width field encodings and explicit length fields for variable-length anchor references. Canonical encoding is not merely an implementation detail; it is part of the cryptographic construction.

6.3 Payload and Record Binding

Lemma 1 (Payload binding)**.** *Assume second-preimage resistance of the payload domains PTXT and CTXT. For a fixed payload interpretation $e \in \{0,1\}$, given a payload P, it is infeasible to find $P' \neq P$ such that $\mathsf{PayloadCommit}(P, e) = \mathsf{PayloadCommit}(P', e)$.*

Proof. For $e = 0$, the claim is second-preimage resistance of $\mathsf{cSHAKE256}_{\mathsf{PTXT}}(\cdot)$. For $e = 1$, it is second-preimage resistance of $\mathsf{cSHAKE256}_{\mathsf{CTXT}}(\cdot)$. The domain is fixed in each case. □

Lemma 2 (Record commitment binding)**.** *Assume canonical record-header encoding and second-preimage resistance of* PTXT*,* CTXT*, and* RCRD*. Given a record* $R = (\mathsf{Hdr}, P)$ *and payload interpretation* e*, it is infeasible to find a distinct record* $R' = (\mathsf{Hdr}', P')$ *with the same interpretation such that*

$$\mathsf{RecordCommit}(R, e) = \mathsf{RecordCommit}(R', e).$$

Proof. The record commitment input is $\mathsf{Enc}(\mathsf{Hdr}) \parallel \mathsf{PCom}$. If $\mathsf{Hdr} \neq \mathsf{Hdr}'$, canonical encoding gives different header encodings. If $P \neq P'$, Lemma 1 gives a different payload commitment except with negligible probability. Equality of record commitments on distinct inputs then gives a second preimage or collision in the RCRD domain. □

6.4 Merkle Root Binding

Lemma 3 (Node binding)**.** *Assume collision resistance of the* NODE *domain. It is infeasible to find two distinct ordered pairs* $(x, y) \neq (x', y')$ *of 32-byte values such that*

$$\mathsf{Node}(x, y) = \mathsf{Node}(x', y').$$

Proof. The node input has fixed length 64 bytes and is parsed uniquely as two 32-byte values. Equality of node outputs for distinct pairs gives a collision in $\mathsf{cSHAKE256}_{\mathsf{NODE}}(\cdot)$. □

Lemma 4 (Merkle root binding)**.** *Assume collision resistance of the* NODE *domain and binding of leaf values. It is infeasible for a PPT adversary to find two distinct ordered leaf sequences* $\mathbf{x} \neq \mathbf{x}'$ *such that*

$$\mathsf{MerkleRoot}(\mathbf{x}) = \mathsf{MerkleRoot}(\mathbf{x}').$$

Proof. The proof is the standard Merkle binding argument. If two distinct trees have the same root, traverse downward from the root to the first node at which the represented subtrees differ while the node value is equal. If the node is internal, this gives two distinct child pairs with the same NODE hash, contradicting Lemma 3. If the node is a leaf, it gives equality of distinct leaf commitments, contradicting leaf binding. The duplicate-last rule is deterministic, so the tree shape is uniquely determined by the leaf count. □

6.5 Block Soundness

Theorem 1 (Block soundness)**.** *Assume record commitment binding and Merkle root binding. A block Merkle root uniquely binds the ordered sequence of record commitments in that block except with negligible probability.*

Proof. If an adversary can produce two different ordered record-commitment sequences with the same block root, it violates Lemma 4. If it substitutes a different record while preserving a leaf commitment, it violates record commitment binding. □

Theorem 2 (Block commitment binding)**.** *Assume canonical block-header encoding and collision resistance of* BLCK*. It is infeasible to produce two distinct pairs* $(\mathsf{BHdr}, \mathsf{BRoot}) \neq (\mathsf{BHdr}', \mathsf{BRoot}')$ *with the same block commitment.*

Proof. The block commitment input is $\mathsf{Enc}(\mathsf{BHdr}) \parallel \mathsf{BRoot}$. The header encoding is canonical and BRoot has fixed length. Distinct pairs produce distinct inputs. Equality of outputs gives a collision in the BLCK domain. □

6.6 Checkpoint-Chain Prefix Uniqueness

Definition 2 (Checkpoint chain)**.** *A checkpoint chain of length t is a sequence $(Q_1, \ldots, Q_t)$ such that each checkpoint commitment satisfies*

$$\mathsf{QCom}_j = \mathsf{cSHAKE256}_{\mathsf{CHCK}}\big(\mathsf{Enc}(\mathsf{QHdr}_j) \parallel \mathsf{BRoot}_j \parallel \mathsf{QCom}_{j-1}\big)$$

with $\mathsf{QCom}_0 = 0^{256}$.

Theorem 3 (Checkpoint commitment binding)**.** *Assume canonical checkpoint-header encoding and collision resistance of* CHCK*. It is infeasible to produce two distinct triples* $(\mathsf{QHdr}, \mathsf{BRoot}, \mathsf{Prev})$ *and* $(\mathsf{QHdr}', \mathsf{BRoot}', \mathsf{Prev}')$ *with the same checkpoint commitment.*

Proof. The checkpoint commitment input is the canonical checkpoint header, a fixed-length block root, and a fixed-length previous commitment. The parse is unique. Equal outputs on distinct triples give a collision in CHCK. □

Theorem 4 (Checkpoint-chain prefix uniqueness)**.** *Assume Theorems 1, 2, and 3. If two valid checkpoint chains of the same length t produce the same head commitment at index t, then the two checkpoint chains bind the same sequence of checkpoint headers, block roots, previous commitments, and covered record commitments except with negligible probability.*

Proof. Let the two chains first differ at checkpoint j. If their head commitments are equal, recursively applying Theorem 3 backward from the head commitment shows that the triples at each checkpoint must be equal, including the previous commitment. At the first differing checkpoint this is impossible except by a collision in CHCK. Equality of the bound block roots gives equality of the covered record-commitment sequences by Theorem 1. Equality of record commitments binds the records by Lemma 2. □

Remark 1 (Reference chain-link checks)**.** *Beyond the cryptographic previous-commitment binding, the reference chain-link verifier enforces sequential checkpoint numbering (the checkpoint sequence incremented by one), non-decreasing first-record sequence, and non-decreasing checkpoint timestamps. These are operational ordering checks layered on the commitment binding; they are not required for the prefix-uniqueness reduction, but they cause a verifier to reject chains that violate the expected ordering discipline.*

Corollary 1 (Record modification detectability)**.** *Given a verified checkpoint chain, modifying any record covered by a sealed checkpoint changes either a record commitment, a block root, a block commitment, or a checkpoint commitment, except with negligible probability.*

Proof. A record modification changes the payload commitment or record commitment except with negligible probability. This changes the block Merkle root by Theorem 1. The altered root changes the checkpoint commitment by Theorem 3. □

Corollary 2 (Deletion and reordering detectability)**.** *Deleting or reordering records inside a sealed block, or reordering sealed checkpoints, is detected relative to a known checkpoint head except with negligible probability.*

Proof. Deletion or reordering within a block changes the ordered leaf sequence and therefore the block root by Lemma 4. Reordering checkpoints changes the previous-commitment inputs of the checkpoint chain and violates Theorem 4 unless a collision is found. □

6.7 Inclusion Proof Soundness

Theorem 5 (Inclusion proof soundness)**.** *Assume Merkle root binding. If an inclusion proof verifies for record commitment x at index i under a verified block root M and count n, then x is the leaf at position i in the ordered leaf sequence committed by M, except with negligible probability.*

Proof. Because the verifier derives the path directions from the index i, a verifying proof defines the unique leaf-to-root path for position i with a fixed count. If x were not the committed leaf at position i, then this path and the actual tree would define two distinct ordered leaf sequences with the same root M, contradicting Merkle root binding. □

6.8 Consistency Proof Soundness

Theorem 6 (Consistency proof soundness)**.** *Let $m \leq n$. A deterministic full-leaf consistency proof for roots M_m and M_n verifies only if the first m commitments of the supplied sequence produce M_m and the first n commitments produce M_n.*

Proof. Verification recomputes both Merkle roots from the supplied commitment sequence and compares them to the asserted roots. Acceptance is therefore exactly the conjunction of both root equalities. Soundness follows from Merkle root binding. □

6.9 Signed Checkpoint Authenticity

Theorem 7 (Signed checkpoint authenticity)**.** *Assume the signature scheme is EUF-CMA secure. An adversary that does not possess the signing key cannot produce a valid signature on a new checkpoint commitment except with negligible probability.*

Proof. A forged signed checkpoint commitment is a valid signature on a message not previously signed by the authority. This is directly an EUF-CMA forgery. □

The theorem authenticates the checkpoint commitment, not the truthfulness of the records. A valid signature means that the signing authority attested to the commitment value; it does not mean the record contents were factually correct.

Remark 2. *Reference checkpoint verification recomputes the expected commitment from the encoded checkpoint header, block root, and previous commitment, and compares it against the signature-recovered message before acceptance. The signature therefore authenticates the actual sealed state, not merely an opaque 32-byte value. In the reference profile the checkpoint bundle is always signed, and bundle, head, and audit-path verification require a valid signature.*

6.10 Anchor Binding and Optional Anchor Authenticity

Theorem 8 (Anchor commitment binding)**.** *Assume canonical anchor-reference encoding and collision resistance of* ANCR*. Given an anchor commitment* ACom*, it is infeasible to find two distinct pairs* $(\mathsf{QCom}, \mathsf{Enc}(\mathsf{ARef}))$ *and* $(\mathsf{QCom}', \mathsf{Enc}(\mathsf{ARef}'))$ *that produce the same* ACom*.*

Proof. The anchor commitment input is a fixed-length checkpoint commitment followed by a canonical anchor reference. Distinct pairs imply distinct inputs. Equal anchor commitments imply a collision in ANCR. □

Theorem 9 (Optional signed-anchor authenticity)**.** *If an MCEL deployment profile signs anchor commitments or encoded anchor messages using an EUF-CMA secure signature scheme, then an*

adversary cannot produce a valid signed anchor for a new anchor message except with negligible probability.

Proof. A valid signature on a new anchor message is an EUF-CMA forgery. □

Baseline anchor commitments are unsigned and public to compute. They provide binding integrity but not authenticity. An adversary can compute a syntactically valid anchor commitment for any checkpoint commitment and any encoded reference. The security value of an unsigned anchor arises when the anchor commitment is externally retained, published, timestamped, or compared by verifiers.

6.11 Domain Separation

Theorem 10 (Object-domain isolation)**.** *Assume the cSHAKE customization strings in Table 1 define independent domains. An object commitment computed in one MCEL domain cannot be reinterpreted as a valid commitment in a different MCEL object domain without a cross-domain collision, except with negligible probability.*

Proof. Each object class uses a distinct customization string. Even when message-body bytes are identical, the cSHAKE input domain differs. Equality of outputs across two distinct domains sufficient to pass both object verifiers would violate the assumed independence of the cSHAKE customization domains or yield a cross-domain collision. □

Remark 3. *This theorem concerns MCEL object classes such as records, nodes, blocks, checkpoints, payloads, and anchors. It does not by itself make the operational namespace identifier part of every record commitment. Namespace separation is an operational and storage-context property unless an implementation profile explicitly commits namespace identifiers into its artifact formats.*

7 Security Invariants

Invariant 1 (Record binding)**.** *A sealed record is bound by its payload commitment and record commitment. Any change to the payload or committed header changes the record commitment except with negligible probability.*

Invariant 2 (Block ordering)**.** *A block root binds an ordered sequence of record commitments. Reordering records changes the block root except with negligible probability.*

Invariant 3 (Checkpoint continuity)**.** *Every non-genesis checkpoint commitment binds the previous checkpoint commitment. A verified chain therefore represents a sequence of checkpoint transitions relative to a known or retained checkpoint. Rooting at the distinguished genesis value* 0^{256} *is established only when verification begins from the genesis checkpoint or independently checks that the first previous-commitment equals* 0^{256}*; the reference audit-path verifier checks relative linkage and does not by itself assert the genesis value.*

Invariant 4 (Signature scope)**.** *A checkpoint signature authenticates the checkpoint commitment. It does not directly sign every record, but every record covered by the checkpoint affects the signed commitment through the record commitment, block root, and checkpoint commitment path.*

Invariant 5 (Anchor scope)**.** *An anchor commitment binds a checkpoint commitment to an external reference. It does not authenticate the producer unless an external authentication layer is applied.*

Invariant 6 (Policy independence)**.** *Policy checks constrain which operations a compliant deployment accepts. They do not replace cryptographic verification. A policy-compliant ledger and a policy-violating ledger are both cryptographically analyzable by the same commitment rules.*

8 Correctness Properties

Proposition 1 (Deterministic commitment)**.** *Given identical payload bytes, headers, block roots, previous commitments, and anchor references, all conforming MCEL implementations compute identical payload commitments, record commitments, block commitments, checkpoint commitments, and anchor commitments.*

Proof. All commitment functions are deterministic cSHAKE-256 computations over fixed domain strings and canonical byte encodings. □

Proposition 2 (Deterministic block verification)**.** *Given the ordered record commitments for a block and the encoded block header, a verifier can recompute the block root and block commitment without interaction with the ledger authority.*

Proof. The Merkle construction and block commitment function are deterministic and require only public artifact bytes. □

Proposition 3 (Deterministic checkpoint verification)**.** *Given a checkpoint header, block root, previous checkpoint commitment, and optional signature, a verifier can recompute the checkpoint commitment and verify the signature if present.*

Proof. The checkpoint commitment is deterministic. Signature verification, when used, is a public operation over the checkpoint commitment. □

Proposition 4 (Append-only evidence)**.** *If a verifier retains a checkpoint commitment* QCom_j*, any later presented history that excludes, changes, or reorders records covered by that checkpoint is rejected except with negligible probability.*

Proof. The claim follows from checkpoint-chain prefix uniqueness and block soundness. □

9 Limitations and Non-Goals

9.1 No Consensus

MCEL does not provide distributed agreement. A malicious or compromised authority can produce divergent checkpoint chains. MCEL makes divergence detectable when verifiers compare retained checkpoint commitments, signed attestations, or external anchors. It does not force all parties to see the same head state.

9.2 No Liveness or Availability

MCEL does not guarantee that records are appended, checkpoints are produced, or artifacts remain available. Storage redundancy, publication policies, and operational monitoring are deployment responsibilities.

9.3 No Truthfulness Guarantee

MCEL ensures that records are tamper evident after commitment. It does not prove that a record was truthful, authorized, or complete at the time of creation. Those properties must be supplied by surrounding processes.

9.4 Replay of Older Valid States

An older checkpoint commitment remains cryptographically valid. A verifier that lacks an expected current checkpoint cannot distinguish a stale but valid state from the latest state. Freshness requires retained state references, signed publication, external anchors, or another freshness policy.

9.5 Policy and Identity

MCEL may include a policy layer, but policy enforcement is not the cryptographic root of trust. Identity, authorization, and governance are deployment-specific. MCEL can commit policy records or key-rotation records, but it does not define an institutional authorization framework.

10 Implementation Considerations

10.1 Hash Function Selection

The construction analyzed here uses cSHAKE-256 with output length 256 bits, function-name string `MCEL-LEDGER`, and the customization strings listed in Table 1. Implementations that use different primitives or domain strings produce different artifacts and are not interoperable with this construction. With a 256-bit output, generic classical collision resistance provides approximately 128-bit security by the birthday bound; in quantum-access hash models the generic collision margin is lower. Deployments requiring a larger collision-security margin SHOULD select a longer output through the algorithm-agility mechanism and regenerate the affected commitments at the migration boundary.

10.2 Merkle Tree Rule

The duplicate-last rule for odd node counts must be applied consistently. Promoting an unpaired leaf without hashing, duplicating on the opposite side, or using a different balancing rule produces different roots for the same leaf sequence.

10.3 Key Rotation

A key-rotation record commits new key material or a new key identifier into the ledger. Verifiers require a deployment policy that specifies when a rotation becomes active, whether it affects checkpoint signatures immediately or after a boundary, and how prior keys remain valid for historical checkpoints. The cryptographic ledger can prove that a rotation record was included in a sealed state; it cannot decide whether the rotation was authorized unless that authorization rule is externalized.

10.4 Policy Enforcement

The policy context must be maintained consistently by the caller or higher-level ledger interface. Monotonic sequence and timestamp checks are meaningful only if the policy context reflects the last

accepted state. Policy errors should fail closed. However, even if policy enforcement is disabled, cryptographic verification of produced artifacts remains deterministic.

10.5 Test Vectors

Interoperability requires known-answer vectors for each commitment layer. Appendix A provides representative vector values produced under the cSHAKE-256 domain definitions in this paper. Independent implementations should reproduce these values before attempting cross-system artifact exchange.

11 Related Work

Merkle trees originate in Merkle's work on public-key cryptosystems and authenticated data structures. Haber and Stornetta introduced cryptographic time-stamping of digital documents, establishing a foundation for tamper-evident historical records. Certificate Transparency, specified in RFC 6962 and updated in later work, uses Merkle trees and signed tree heads to make certificate issuance auditable. MCEL differs from Certificate Transparency by targeting controlled-writer evidence ledgers rather than public certificate logs, and by binding sealed blocks into an explicit checkpoint commitment chain.

Crosby and Wallach studied efficient tamper-evident logging, including authenticated data structures for audit logs. MCEL is aligned with that line of work but uses cSHAKE-based domain separation and a layered record-block-checkpoint structure. The use of cSHAKE-256 follows NIST SP 800-185, and the checkpoint signature, mandatory in the reference construction, is instantiated by default with ML-DSA as standardized in FIPS 204.

12 Conclusion

MCEL is a conservative append-only evidence ledger construction built from canonical serialization, domain-separated cSHAKE-256 commitments, deterministic Merkle aggregation, and checkpoint chaining. Its security properties are narrow and explicit. Records are bound by payload and record commitments; blocks bind ordered record commitments through a Merkle root; checkpoints bind block roots into a cumulative chain; signed checkpoint bundles authenticate checkpoint commitments, with signing mandatory in the reference profile; anchors bind checkpoint commitments to external references.

The analysis shows that undetected modification, deletion, or reordering of records covered by a verified checkpoint requires a violation of canonical encoding, collision resistance, second-preimage resistance, Merkle binding, or signature unforgeability when signatures are used. The construction does not provide consensus, truthfulness, liveness, or automatic freshness. Those properties require external governance and deployment policy. Within its intended non-consensus setting, MCEL provides a compact, deterministic, and independently verifiable basis for evidentiary record keeping.

A Representative Commitment Vectors

This appendix gives representative commitment outputs for a fixed test harness. The values are hexadecimal encodings of 32-byte outputs. They are included to support conformance testing and to make the domain-separated commitment layers explicit.

Name	Hex value
Plaintext payload commitment	4BC150A0B2D5AB42 37F96C8F9AA98967 614B3267A7AE5AEA 8C85DED536C56837
Ciphertext payload commitment	4E696682287736EA 34908457E88E13D2 2245F643CD394F99 FEEE3D052ECA9939
Record commitment	001FA935D577149E B02714F4F34BFCF6 ACFE5EC94F59F74C 4DCE2EFA4DF3EEEA
Block Merkle root	3D47503411CA16CC 2C92F48595EAC178 58E1906B5D5D58EF 5462AFD73F4583E3
Block commitment	C6B7A041B2E2D6D1 63A46CF4C9C4D46D 43CD49CB24E20B53 642191E6A32422D1
Checkpoint commitment	07C577CD8DDE8C66 3D2C600CB81D1906 494820ECC31CBA33 3186F155335D53AA
Anchor commitment	602E05687CCE36C9 0C623314448D26E6 EBA10D8A9356BDF4 EC5B9B2EF05A2645

References

[1] R. C. Merkle, *Protocols for Public Key Cryptosystems*, IEEE Symposium on Security and Privacy, 1980. https://ieeexplore.ieee.org/document/6233529

[2] S. Haber and W. S. Stornetta, *How to Time-Stamp a Digital Document*, Journal of Cryptology, 3(2), 1991. https://link.springer.com/article/10.1007/BF00196791

[3] S. A. Crosby and D. S. Wallach, *Efficient Data Structures for Tamper-Evident Logging*, USENIX Security Symposium, 2009. https://www.usenix.org/legacy/event/sec09/tech/full_papers/crosby.pdf

[4] A. Langley, B. Laurie, E. Kasper, and R. Stradling, *Certificate Transparency*, RFC 6962, Internet Engineering Task Force, 2013. https://datatracker.ietf.org/doc/html/rfc6962

[5] National Institute of Standards and Technology, *SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions*, FIPS PUB 202, 2015. https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.202.pdf

[6] J. Kelsey, S. Chang, and R. Perlner, *SHA-3 Derived Functions: cSHAKE, KMAC, TupleHash, and ParallelHash*, NIST Special Publication 800-185, 2016. https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-185.pdf

[7] National Institute of Standards and Technology, *Module-Lattice-Based Digital Signature Standard*, FIPS PUB 204, 2024. https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.204.pdf

[8] M. Bellare and P. Rogaway, *Introduction to Modern Cryptography*, UCSD Lecture Notes, 2005. https://cseweb.ucsd.edu/~mihir/papers/intro.pdf

[9] J. G. Underhill, *Merkle-Chained Evidence Ledger - MCEL 1.0 Technical Specification*, Quantum Resistant Cryptographic Solutions Corporation. https://www.qrcscorp.ca/documents/mcel_specification.pdf